Unit 3

Synchronization and Deadlock

Process Synchronization

Definition of Process

- A process is a program in execution, characterized by:
    - **State Information:** Current status, including program counter and resources.
    - **Resources:** Memory usage, open files, and allocated I/O devices.
    - **Execution Context:** Includes the state of registers, counters, and process control block (PCB).

Types of Programs

1. **Concurrent Program:**
    - A program with more than one execution context.
    - It can execute multiple tasks or threads independently or with synchronization.
2. **Parallel Program:**
    - A special type of concurrent program.
    - Executes multiple threads or tasks simultaneously.
3. **Cooperating Concurrent Processes:**
    - Processes working together, sharing information and coordinating tasks.
    - Example: Concurrent C programs.

Distinguishing Process from Application

- A process can represent a single running program or multiple cooperating processes appearing as a single application to the user.

Concurrency

Definition:

Concurrency refers to the execution of multiple control flows (processes or threads) overlapping in time, where these flows may need to interact or share resources.

Examples of Concurrency:

Hardware Exception Handlers: Handle exceptions during the execution of processes.

Processes: Programs in execution managed by operating systems.

Importance of Concurrency:

- Simultaneous execution of multiple programs.
- Background execution of tasks.
- Efficient use of system resources.

Problems Caused by Concurrency:

- Access to Shared Data Structures: Concurrent access can cause data inconsistency.
- Deadlock: Processes get stuck waiting for each other due to resource contention.
- Complex Process Interaction: Difficult to synchronize and manage communication between processes.

Concurrency vs. Parallelism:

Concurrency:

- Multiple tasks/processes execute within the same time frame but not necessarily at the same moment.
- There may be dependencies between tasks.

Parallelism:

- Multiple tasks execute simultaneously, often on different cores or processors.
- Focused on independent calculations to speed up processing.
- Key Difference: Concurrency is a problem scenario, whereas parallelism is a solution technique.

Types of Resources:

Reusable Resources:

Resources that can be used multiple times without depletion.

Examples:

CPU: Executes instructions.

Main Memory: Stores data for processes.

I/O Devices: Handle input/output operations.

Consumable Resources:

Resources that are created by one process and consumed by another.

Examples:

Message Queues: A process sends a message that another process consumes.

Synchronization Events: Notifications between processes.

Resource Competition in Process Synchronization:

CPU Competition: Processes compete for execution time.

Memory Competition: Multiple processes request access to main memory.

I/O Device Competition: Processes require input/output device allocation.

Secondary Storage: Access to file systems or storage areas.

Data Structures in Main Memory: Sharing and synchronization of data between processes.

Principle of Concurrency

- Concurrent access to shared data may result in data inconsistency. Maintaining data consistency requires mechanisms to ensure the orderly execution of cooperating processes.
- Concurrency arises in the same way at different levels of execution streams. Following are examples of concurrency in different types of operating systems:

1. **Concurrency in Multiprogramming:**
    - Interaction between multiple processes running on one CPU.
2. **Concurrency in Multithreading:**
    - Interaction between multiple threads running in one process.
3. **Concurrency in Multiprocessors:**
    - Interaction between multiple CPUs running multiple processes or threads.
4. **Concurrency in Multi-Computers:**

    - Interaction between multiple computers running distributed processes or threads.

- Java is a concurrent programming language.
- **Process synchronization** is required in:
    - Uni-processor systems
    - Multiprocessor systems
    - Networked systems.

If more than one thread exists in a system at the same time, then the threads are said to be concurrent. Synchronization problems can occur whenever two or more concurrent processes use any shared resource.

Race Condition

- Occurs when two or more operations are performed concurrently on shared data, and the final outcome depends on the timing or sequence of the operations.
- The result becomes unpredictable, leading to inconsistencies.

Key Characteristics

- **Order Dependency:** The outcome depends on the order in which processes or threads are executed.
- **CPU Scheduling:** The operating system's scheduling determines the interleaving of operations, which can cause non-deterministic behavior.
- **Non-Determinism:** Different executions on the same input may produce different results.

Why Race Conditions Must Be Avoided

- **Data Inconsistency:** May corrupt shared data.
- **Hard to Debug:** Difficult to identify and reproduce errors since the issue depends on precise execution timing.
- **Unreliable Applications:** Can cause unexpected behavior in critical systems.

Examples

1. **Banking System Scenario:**
    - Shared variable: balance

- **Operation 1:** Bank employee deposits an amount: balance = balance + amount
- **Operation 2:** Simultaneously, a transfer operation deducts an amount: balance= balance - amount
- **Problem:** If operations execute concurrently without proper synchronization, the final value of balance may be incorrect.

2. **Counter Increment:**
   - Two threads increment a shared counter simultaneously.
   - Without synchronization, both threads may read the same initial value and write incorrect results.

Solutions to Avoid Race Conditions

1. **Synchronization Mechanisms:**
   - Use locks, semaphores, and monitors to control access to shared resources.
2. **Atomic Operations:**
   - Ensure that read-modify-write operations are performed as a single, indivisible unit.
3. **Critical Sections:**
   - Identify code segments that require exclusive access to shared resources.

Critical Section Problem:

In any system where multiple processes or threads operate concurrently, there is a possibility that they will need to access or modify shared resources—such as common variables, files, memory locations, or even system tables. When such shared resources are involved, there is a risk of data inconsistency or corruption if more than one process accesses these resources simultaneously. This challenge is known as the *Critical Section Problem*.

A **critical section** is defined as a block of code where a process accesses shared, modifiable data. Only one process should be allowed to execute its critical section at any given time to ensure that the shared data remains consistent. The fundamental goal is to design a protocol that guarantees mutual exclusion, meaning that while one process is executing its critical section, no other process can enter its own critical section for the same shared resource.

Structure of a Process with Critical Sections:

To address this problem, processes are generally structured into four distinct parts:

1. **Entry Section:**
   This is the initial segment of the code where the process requests permission to enter its critical section. Here, the process checks whether it is safe to proceed into the critical section based on the status of other processes.
2. **Critical Section:**
   This is the block of code where the process accesses and manipulates shared resources. Since the data here is susceptible to concurrent modifications, it is imperative that only one process can be executing in its critical section at any moment.
3. **Exit Section:**
   After completing the operations in the critical section, the process executes the exit section. This segment is responsible for releasing the exclusive access (or mutual exclusion) so that other processes waiting to enter their critical sections can proceed.
4. **Remainder Section:**
   This consists of the rest of the code that does not involve accessing shared resources. Processes spend most of their execution time in this section where no special synchronization is required.

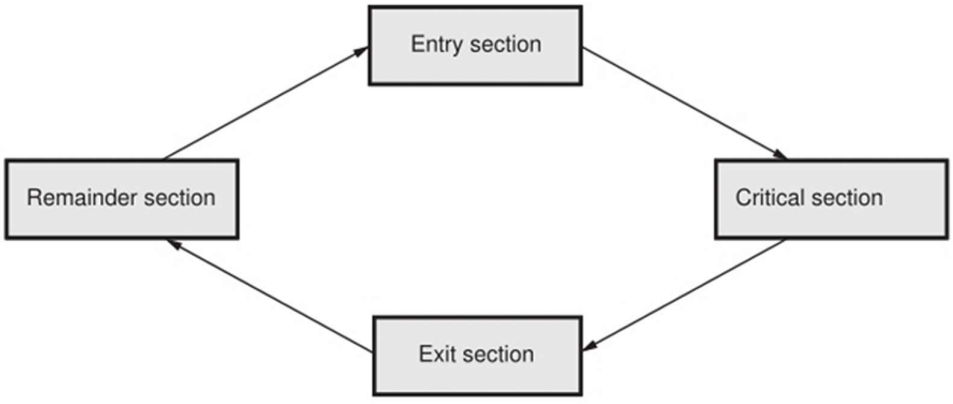
Every process typically cycles through these sections in the following order: remainder, entry, critical, and exit.

Illustrative Example

Consider a simple scenario in which a shared variable, say count, is initialized to 5. Two processes perform the following operations concurrently:

- **Process 1:** Increases the count by 1 (i.e., count = count + 1).
- **Process 2:** Decreases the count by 1 (i.e., count = count - 1).

Due to the lack of proper synchronization, the final value of count can vary (it might be 6, 5, or even another value) depending on the precise timing and interleaving of these operations. This unpredictability is a classic manifestation of the race condition within a critical section.



Solution to Critical Section Problem:

To solve the critical section problem, any solution must satisfy the following three requirements:

1. **Mutual Exclusion:**
    - Only one process can execute in its critical section at a time for a particular shared resource.
    - If a process is in its critical section, other processes attempting to enter their critical sections must wait.
2. **Progress:**
    - If no process is in the critical section and some processes wish to enter, only those processes not in their remainder section should participate in the decision of who enters next.
    - This selection must not be postponed indefinitely.
3. **Bounded Waiting:**
    - There must be a finite limit on the number of times other processes are allowed to enter their critical sections after a process has requested entry.
    - This prevents starvation of any process.

Software and Hardware Solutions

Software Solutions:

These involve implementing synchronization algorithms using software techniques:

1. **Peterson's Algorithm:**
    - A classic solution for two-process synchronization.
    - Uses two shared variables: flag (indicates if a process wants to enter the critical section) and turn (determines whose turn it is to enter).
2. **Semaphore Mechanism:**
    - A synchronization primitive provided by operating systems.
    - Two types:
        - **Counting Semaphore:** Manages access to multiple resources.
        - **Binary Semaphore (Mutex):** Manages access to a single shared resource.

- Semaphores provide functions wait() (P) and signal() (V) to control access.

3. **Monitor Constructs:**
    - High-level synchronization construct.
    - Monitors encapsulate shared variables, procedures, and synchronization mechanisms.

Hardware Solutions:

1. **Disabling Interrupts:**
    - A process disables interrupts during its critical section to ensure uninterrupted execution.
2. **Test-and-Set Instruction:**
    - An atomic hardware operation used to create mutual exclusion.
3. **Compare-and-Swap Instruction:**
    - Compares the contents of a memory location with a given value and swaps them if they match, ensuring atomicity.

Kernel-Based Solutions

Operating systems handle critical section problems using the kernel, which can be:

- **Preemptive Kernel:**
    - Allows preemption even when a process is in kernel mode.
    - Requires sophisticated synchronization mechanisms.
- **Non-Preemptive Kernel:**
    - Does not allow preemption during kernel operations.
    - Easier to implement synchronization as processes run to completion in kernel mode.

Critical Region:

- Writing or reading a set of data into a section of memory generally takes some time and is performed through the execution of a series of primitive operations.
- A critical region is an area of a process sensitive to inter-process complications. To guarantee mutual exclusion, only one process is allowed to enter its critical region at a time.

- **Operating System Requirement:** The operating system needs a system to ensure that a process cannot enter its critical region if another process is already in it.
- **Exclusive Access:** A process must be allowed to finish work on a critical part of the program before other processes can have access to it. It is called a critical region because it is a critical section, and its execution must be handled as a unit. Other processes must wait before accessing critical region resources.
- **Conditional Critical Region:** Each conditional critical region is a "resource" that has a name and a set of variables that can only be accessed in that region. All variables can only be accessed within a conditional critical region.

Mutual Exclusion:

Mutual exclusion is a key concept in concurrent programming that ensures only one process or thread can access a shared resource or critical section at any given time. This prevents data corruption and ensures consistency in multi-threaded or multi-process systems.

Need for Mutual Exclusion:

Prevent Data Corruption: Ensures shared resources are accessed in a consistent manner.

Synchronization: Coordinates execution among concurrent processes to avoid conflicts.

Fair Resource Sharing: Prevents monopolization of shared resources by any single process.

Avoiding Race Conditions: Eliminates unpredictable outcomes when multiple processes access shared data simultaneously.

Requirements of Mutual Exclusion:

1. **Mutual Exclusion:** Only one process can access the critical section at a time.
2. **Progress:** If no process is in the critical section, then only the processes that want to enter the critical section should participate in the decision of which process will enter next.
3. **Bounded Waiting:** Each process must have a limited waiting time to enter the critical section.
4. **Freedom from Deadlock:** No situation where processes block each other indefinitely.
5. **Speed Independence:** The solution should work correctly regardless of the relative speeds of the processes.

Approaches to Achieve Mutual Exclusion

1. **Software Solutions:**
   - Implemented purely through algorithms without hardware support.
   - Examples:

        - **Peterson's Algorithm**: Ensures mutual exclusion using flags and turn variables.
        - **Dekker's Algorithm**: One of the earliest mutual exclusion solutions for two processes.
2. **Hardware Solutions:**
    - Use special atomic machine instructions to manage access to shared resources.
    - Examples:
        - **Test-and-Set**: Atomically sets a value and returns the previous one.
        - **Compare-and-Swap:** Compares and updates memory contents.
3. **Operating System Support:**
    - Use synchronization primitives provided by the operating system.
    - Examples:
        - **Mutex (Mutual Exclusion Locks):** Locks that prevent simultaneous access.
        - **Semaphores:** Counting variables that control access to resources.
        - **Monitors:** High-level synchronization constructs that manage access to shared data.
4. **Programming Language Support:**
    - Modern programming languages provide built-in synchronization mechanisms.
    - Examples:
        - **Java:** synchronized keyword.
        - **Python:** threading.Lock.

Challenges in Implementing Mutual Exclusion

- **Deadlock**: Occurs when two or more processes wait indefinitely for resources held by each other.
- **Starvation:** A process may never get access to the critical section due to higher-priority processes.
- **Fairness**: Ensuring that all processes have an equal chance to access shared resources
- **Complexity**: Designing correct and efficient mutual exclusion mechanisms is challenging.

Lock in Process Synchronization

A **lock** is a synchronization mechanism used to control access to shared resources in concurrent programming environments. Locks help ensure that only one process or thread accesses a critical section at a time.

Lock Mechanism

- **Lock = 0:** Indicates that no process is in the critical section (the section is free to be acquired).
- **Lock = 1:** Indicates that a process is currently in the critical section (the section is locked).

Locks are a simple mechanism but come with certain challenges:

- **Synchronization Problem:** Using simple lock variables does not completely solve process synchronization issues due to the problem of race conditions.
- **Busy Waiting:** If multiple processes continuously check for the lock to become free, it results in wastage of CPU cycles.

Mutex (Mutual Exclusion Object)

A **Mutex (Mutual Exclusion)** is a synchronization mechanism used in concurrent programming to allow only one thread or process at a time to access a shared resource, such as memory, files, or critical sections.

Key Characteristics of Mutex

1. **Ownership:**
   The process or thread that locks the mutex must be the one that unlocks it. This ensures accountability and prevents arbitrary unlock operations.
2. **Binary State:**
   A mutex can only have two states:
   - **Locked (1):** Indicates that the critical section is currently occupied by a process.
   - **Unlocked (0):** Indicates that the critical section is available for access.
3. **Blocking Behavior:**
   - If the mutex is locked, the calling process goes into a waiting state until the mutex becomes available.
   - Mutex provides **blocking semantics**, meaning waiting processes do not continuously poll for the resource, saving CPU resources.

How Mutex Works

1. **Request for Mutex:**
   - A process attempts to lock the mutex before entering its critical section.
2. **Check Mutex Status:**
   - If the mutex is **unlocked (0)**, the process locks it and enters the critical section.
   - If the mutex is **locked (1)**, the process is **blocked** until the mutex becomes available.
3. **Release Mutex:**
   - After completing the task in the critical section, the process unlocks the mutex, allowing other waiting processes to access the resource

Advantages of Mutex

1. **Prevents Race Conditions:** Mutex ensures mutual exclusion, preventing simultaneous access to shared resources.
2. **Efficient Resource Usage:** Avoids CPU wastage by blocking waiting processes instead of keeping them in busy loops.

3. **Simple and Lightweight:** Binary implementation makes it easy to manage resource access.
4. **Reliable Synchronization:** Effective for controlling shared resource access in multi-threaded applications.

Limitations of Mutex

1. **Deadlock Possibility:** If a process forgets to release the mutex, it can cause deadlock.
2. **Starvation:** High-priority processes may block low-priority ones indefinitely.
3. **Limited Scope:** Primarily suited for controlling entries to critical regions.

Difference Between Mutex and Binary Semaphore

Aspect	Mutex	Binary Semaphore
Ownership	Owned by the locking process	No specific ownership requirement
Unlocking	Only the owner can unlock	Any process can unlock
Blocking Behavior	Blocks when locked	Can signal between processes
Use Case	Mutual exclusion	Synchronization between processe

Semaphores

Semaphores are synchronization tools used in operating systems to control access to shared resources by multiple processes and solve critical section problems. Introduced by E.W. Dijkstra, semaphores ensure that concurrent processes do not face race conditions while accessing shared data.

Characteristics of Semaphores

- A semaphore is an integer variable used as a signaling mechanism.
- It allows processes to communicate and synchronize their actions.
- The value of the semaphore is never negative.
- Semaphores can either be binary (0 or 1) or counting (with unrestricted integer values).

Semaphore Operations

Semaphores use two atomic operations: **wait (P)** and **signal (V)**.

1)Wait (P) Operation:

- This operation is used to request access to a critical section.
- If the semaphore value is greater than 0, the process decrements it and enters the critical section. If it is 0 or negative, the process blocks itself until it becomes positive.

```
wait(S):
    while S <= 0 do wait;
    S = S - 1;
```

2)Signal (V) Operation:

This operation increments the semaphore value, signaling that the process has exited the critical section.

```
signal(S):
    S = S + 1;
```

Types of Semaphores

1. Binary Semaphore (Mutex)

A **Binary Semaphore**, also known as a **Mutex (Mutual Exclusion)**, is a synchronization mechanism used to protect critical sections in concurrent programming.

Characteristics:

- Can only take two values: **0 (locked)** or **1 (unlocked)**.
- Used primarily for mutual exclusion to ensure that only one process accesses a critical section at a time.
- Lightweight and efficient in managing exclusive access to shared resources.

How it Works:

- When a process wants access to a critical section, it performs a **wait (P)** operation.
- If the semaphore value is **1**, it allows the process to enter and sets the value to **0** (lock).
- When the process exits the critical section, it performs a **signal (V)** operation, resetting the semaphore back to **1** (unlock).

Use Cases:

- File I/O operations, where only one process should access the file at a given time.
- Synchronizing threads in multithreaded applications.

Advantages:

- Simple to implement and understand.
- Efficient for scenarios where resource contention is low.

Disadvantages:

- Not suitable for managing access to resources with multiple instances.
- If not used properly, it can lead to **deadlocks** or **priority inversions**.

2. Counting Semaphore

A **Counting Semaphore** is used when multiple instances of a resource are available, and it is necessary to control access to those resources.

Characteristics:

- Can take any non-negative integer value, representing the number of available resource instances.
- Unlike binary semaphores, counting semaphores do not strictly enforce mutual exclusion.

How it Works:

- The semaphore value is initialized to the total number of available resource instances.
- Each **wait (P)** operation decreases the semaphore value by **1**, indicating that a resource instance is being used.
- Each **signal (V)** operation increases the value by **1**, indicating that a resource instance is released.

Use Cases:

- Managing access to a pool of database connections.
- Controlling access to shared I/O devices with multiple channels.
- Allocating memory blocks in operating systems.

Advantages:

- Efficient for managing multiple instances of shared resources.
- Prevents resource starvation by controlling access order.

Disadvantages:

- More complex to implement compared to binary semaphores.

- Incorrect usage can lead to synchronization issues such as resource leaks.

Comparison Between Binary and Counting Semaphores

Aspect	Binary Semaphore (Mutex)	Counting Semaphore
Value Range	0 or 1	Non-negative integer
Purpose	Mutual exclusion for a single resource	Managing access to multiple instances of a resource
Complexity	Simple and lightweight	More complex to implement
Efficiency	High for scenarios with low resource contention	Efficient for managing multiple resource instances
Concurrency Control	Allows only one process/thread at a time	Allows multiple processes/threads based on the resource count
Resource Allocation	One resource instance at a time	Multiple resource instances simultaneously
Implementation	Easier to implement	Requires more logic to handle resource count
Blocking Mechanism	Immediate blocking when the resource is unavailable	Process is queued until a resource becomes available
Use Case Examples	File I/O operations, protecting shared variables	Database connection pools, memory block allocation
State Management	Two states: Locked (0) and Unlocked (1)	Dynamic states depending on resource availability
Deadlock Risk	Moderate, especially in nested locks	Higher if mismanaged due to complex resource dependencies
Priority Inversion Risk	Common without priority inheritance	Lower but possible with shared resources
Resource Utilization	Suitable for scenarios with low resource demand	Better resource utilization for high-demand scenarios
Scalability	Limited for single-resource access	Scales well for multi-resource environments
Performance Overhead	Minimal	Higher due to resource counting and queuing mechanisms

Properties of Semaphores:

1. Machine-independent and adaptable for various systems.
2. Simple and easy to implement.
3. Help in maintaining synchronization between processes.
4. Can simultaneously acquire multiple resources.
5. Allow different critical sections to be managed by different semaphores.

Drawbacks of Semaphores:

1. **Global Variable Dependency**: Semaphores are shared global variables, which can create programming challenges.
2. **Uncontrolled Access:** Processes can access semaphores from anywhere in the program, leading to misuse.
3. **Complexity**: Ensuring proper usage requires careful programming.
4. **No Direct Association**: There is no explicit link between the semaphore and the resource it controls.
5. **Dual Role:** Semaphores handle both mutual exclusion and scheduling constraints, which can be confusing.

Semaphore functions in C language:

In C programming, semaphores are typically implemented using the POSIX semaphore library (semaphore.h). Below are common functions and their details for semaphore operations in C:

1. sem_init():

Purpose: Initializes a semaphore**.**

Syntax:

```
int sem_init(sem_t *sem, int pshared, unsigned int value);
```

- **Parameters:**
  - sem: Pointer to the semaphore variable.
  - pshared: If 0, the semaphore is shared between threads in the same process. If non-zero, shared between processes.
  - value: Initial value for the semaphore.

Return Value: Returns 0 on success; -1 on failure.

Example:

```
sem_t mutex;
sem_init(&mutex, 0, 1);  // Initialize mutex semaphore with initial value 1
```

2. sem_wait():

Purpose: Decrements (locks) the semaphore. If the semaphore value is 0, the calling process/thread blocks until it becomes positive.

Syntax:

```
int sem_wait(sem_t *sem);
```

- **Parameter:** sem is the pointer to the semaphore.

Return Value: Returns 0 on success; -1 on failure.

Example:

```
sem_wait(&mutex);  // Enter critical section
// Critical section code here
```

3. sem_post()

Purpose: Increments (unlocks) the semaphore. If other processes/threads are waiting, one is woken up.

Syntax:

```
int sem_post(sem_t *sem);
```

- **Parameter:** sem is the pointer to the semaphore.

Return Value: Returns 0 on success; -1 on failure.

Example:

```
sem_post(&mutex);  // Exit critical section
```

4. sem_destroy()

Purpose: Destroys the semaphore and frees resources allocated to it.

Syntax:

```
int sem_destroy(sem_t *sem);
```

- **Parameter:** sem is the pointer to the semaphore.

Return Value: Returns 0 on success; -1 on failure.

Example:

```
sem_destroy(&mutex);  // Clean up the semaphore
```

Monitors in Operating Systems

Monitors are a high-level synchronization construct used in concurrent programming to manage access to shared resources and avoid race conditions. They provide a structured way to enforce mutual exclusion and condition synchronization.

Key Features of Monitors

1. **Encapsulation:**
    - A monitor encapsulates shared variables, data, and operations (functions) that manipulate the shared data.
    - Only monitor procedures can access the shared data, ensuring data protection.
2. **Mutual Exclusion:**
    - Only one process or thread can execute a monitor function at a time.
    - This prevents race conditions and ensures safe concurrent access.
3. **Condition Variables:**
    - Monitors use condition variables to handle process synchronization.
    - Condition variables allow threads to wait for certain conditions to be met before continuing execution.

Components of a Monitor

1. **Shared Variables:**
    - Data shared among processes or threads.
2. **Procedures (Monitor Functions):**
    - Defined operations on the shared data.

3. **Condition Variables:**
    - Used for synchronization between processes.
    - Supports two operations:
        - **wait(condition_variable):** Blocks the current process until the condition variable is signaled.
        - **signal(condition_variable):** Wakes up a process waiting on the condition variable.
4. **Lock Mechanism:**
    - Ensures that only one process is executing inside the monitor at a time.

Example Structure of a Monitor:

```
monitor ExampleMonitor {
   int shared_data;

   condition condition_var;

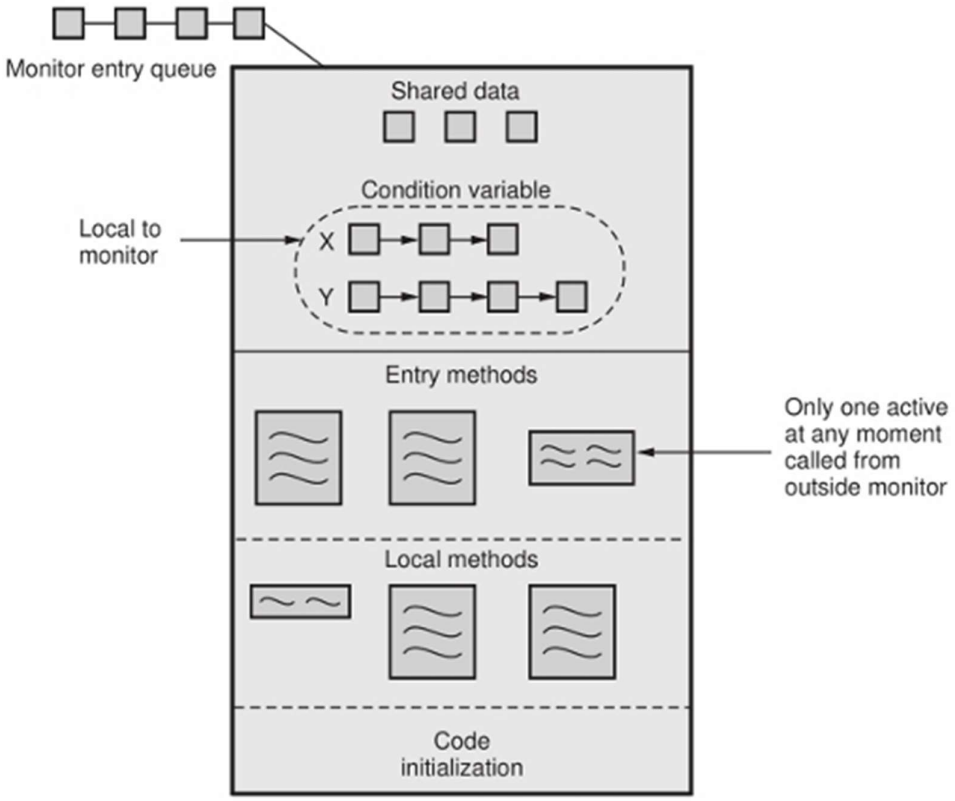
  void procedure1() {
      wait(condition_var);  // Wait for condition
      // Critical section
      signal(condition_var);  // Notify other processes
   }
   void procedure2() {
      // Critical section
   }
}
```

Condition Variable Operations

1. **wait(condition_variable)**
    - Releases the monitor lock.

- Puts the process in a waiting state on the condition variable.

2. **signal(condition_variable)**
   - Wakes up one process waiting on the condition variable.
   - If no process is waiting, the signal is lost.



Advantages of Monitors

- **Simplified Synchronization:** Encapsulation makes synchronization easier and cleaner.
- **Data Encapsulation:** Shared resources are protected within the monitor.
- **Automatic Mutual Exclusion:** Eliminates the need for explicit lock mechanisms.

Disadvantages of Monitors

- **Limited Flexibility:** Less control compared to semaphores for complex synchronization scenarios.
- **Language Support:** Not all programming languages provide built-in support for monitors.

- **Overhead:** Slightly more overhead than semaphores due to abstraction.

Comparison Between Monitor and Semaphore

Aspect	Monitors	Semaphores
Definition	High-level synchronization construct	Low-level synchronization primitive
Data Encapsulation	Encapsulates shared variables and operations	No encapsulation; shared across processes
Synchronization	Built-in mutual exclusion and condition synchronization	Requires explicit synchronization
Complexity	Simplifies coding for process synchronization	Complex and error-prone
Mutual Exclusion	Automatic within monitor functions	Must be explicitly programmed
Condition Handling	Uses condition variables (wait, signal)	Uses wait and signal without explicit condition handling
Blocking Mechanism	Automatically blocks processes on wait()	Requires explicit blocking by the programmer
Implementation	Provided by some programming languages (like Java)	OS or custom code-based implementation
Efficiency	Slightly higher overhead due to abstraction	Lower overhead if managed properly
Concurrency Control	Ensures structured access to shared resources	Less structured, can lead to race conditions
Ease of Use	Easier and safer to use	Prone to misuse and synchronization issues

Classical Synchronization Problems

Reader-Writer Problem

The Reader-Writer Problem occurs when multiple processes require access to a shared resource, such as a file or database, simultaneously. These processes are categorized into two types:

Types of Processes:

- **Readers:**
  - Processes that only read data from the shared resource without modifying it.
  - Multiple readers can access the resource concurrently without issues.
- **Writers:**
  - Processes that modify or update the shared resource.
  - Writers require exclusive access to the resource to prevent data inconsistency.

Problem Characteristics:

- When **only readers** are accessing the resource, they can work concurrently without conflict.
- If a **writer wants access**, it must have exclusive control to ensure data integrity.
- No **reader or writer** is allowed to access the resource while a writer is active.

Example Scenario:

In a **Railway Reservation System:**

- **Readers:** Users viewing the train schedule. Multiple users can read the schedule simultaneously.
- **Writers:** Users making a reservation. Writers must have exclusive access to modify the reservation system database.

Challenges:

1. **Data Inconsistency:**
   - If multiple writers or a combination of readers and writers access the resource simultaneously, it can lead to corrupted or inconsistent data.
2. **Starvation:**

- **Reader Priority:** Writers may be indefinitely delayed if readers continuously access the resource.
- **Writer Priority:** Readers may face starvation if writers are given constant preference.

3. **Deadlock:**
   - If synchronization mechanisms are poorly implemented, processes may block each other indefinitely.

Priority Solutions:

A)First Readers-Writers Problem (Reader Priority):

- Multiple readers can access the resource simultaneously.
- Writers must wait until all readers have completed their operations.
- **Disadvantage:** Writers may starve if there is a continuous stream of readers.

B)Second Readers-Writers Problem (Writer Priority):

- Writers are given priority over readers.
- Once a writer requests access, no additional readers are allowed until the writer finishes.
- **Disadvantage:** Readers may face starvation.

C)Fair Solution (No Starvation):

- Both readers and writers are treated fairly.
- Access is granted based on the order in which requests arrive.

Synchronization Mechanisms:

1. **Semaphores:**
   Used to control access to the shared resource by maintaining counters for readers and writers.
2. **Mutex**
   Ensure mutual exclusion by allowing only one writer or multiple readers at a time.
3. **Monitors:**
   Provide high-level abstraction for synchronization by encapsulating shared resources and operations in a monitor.

Algorithm (while using semaphores):

Data Structures:

- mutex: Binary semaphore for mutual exclusion.
- rw_mutex: Semaphore to control access to the shared resource.
- read_count: Integer to keep track of the number of active readers.

Reader Process:

```
1. wait(mutex)              // Lock to update read_count
2. read_count++
3. if (read_count == 1)
     wait(rw_mutex)         // First reader locks the resource for writers
4. signal(mutex)            // Release lock for other readers
5. Read the resource        // Critical section
6. wait(mutex)              // Lock to update read_count
7. read_count--
8. if (read_count == 0)
     signal(rw_mutex)       // Last reader unlocks the resource for writers
9. signal(mutex)            // Release lock
```

Writer Process:

```
1. wait(rw_mutex)           // Lock resource for exclusive access
2. Write the resource       // Critical section
3. signal(rw_mutex)         // Release resource
```

Producer-Consumer Problem (Bounded Buffer Problem)

The Producer-Consumer problem is a classic synchronization problem in concurrent programming. It deals with processes that share a common, fixed-size buffer for exchanging

data. One or more producer processes generate data and place it into the buffer, while one or more consumer processes retrieve data from the buffer.

Problem Description:

- The **Producer** generates data items and places them into a shared buffer.
- The **Consumer** retrieves data items from the shared buffer and processes them.
- The buffer has a **finite size**, which means it can hold a limited number of data items.
- The challenge is to ensure that:
    1. The producer does not produce data when the buffer is full.
    2. The consumer does not consume data when the buffer is empty.
    3. No two producers or consumers access the buffer simultaneously (mutual exclusion).

Challenges in the Problem:

1. **Synchronization:** Proper synchronization is required to avoid race conditions between producer and consumer processes.
2. **Buffer Overflow:** Preventing the producer from adding data when the buffer is full.
3. **Buffer Underflow:** Preventing the consumer from consuming data when the buffer is empty.
4. **Mutual Exclusion:** Ensuring exclusive access to the buffer during addition or removal of data.

Solution Requirements:

The solution must meet the following criteria:

1. **Mutual Exclusion:** Only one process (producer or consumer) can access the buffer at a time.
2. **Bounded Buffer:** Prevent buffer overflow and underflow.
3. **Deadlock-Free:** The solution must not cause deadlock where processes get stuck waiting indefinitely.
4. **Fairness:** Both producer and consumer should get fair chances to execute without starvation.

Semaphores Used for Synchronization

To solve the problem using semaphores, three semaphores are typically used:

mutex: Ensures mutual exclusion for buffer access (binary semaphore).

empty: Counts the number of empty slots in the buffer (counting semaphore).

full: Counts the number of filled slots in the buffer (counting semaphore).

Algorithm(using semaphores):

Producer Process

```
while (true) {
    wait(empty);              // Wait if buffer is full
    wait(mutex);              // Lock the buffer
    buffer[in] = item;        // Insert item into the buffer
    in = (in + 1) % BUFFER_SIZE; // Update buffer index
    signal(mutex);            // Unlock the buffer
    signal(full);             // Signal that buffer has a new item
}
```

Consumer Process

```
while (true) {
    wait(full);               // Wait if buffer is empty
    wait(mutex);              // Lock the buffer
    item = buffer[out];       // Remove item from buffer
    out = (out + 1) % BUFFER_SIZE; // Update buffer index
    signal(mutex);            // Unlock the buffer
    signal(empty);            // Signal that buffer has an empty slot
    process(item);            // Process the consumed item
}
```

Real-Life Examples of Producer-Consumer Problem

1. **Printing System:** Multiple print jobs (producers) and a single printer (consumer).
2. **Online Streaming:** Video buffering where producers provide frames, and the video player consumes them.
3. **Email Queue:** Emails being produced by senders and consumed by the mail delivery service

Inter-process communication

- Exchange of data between two or more separate, independent processes/threads is possible using IPC. Operating systems provide facilities/resources for IPC, such as message queues, semaphores, and shared memory.
- A complex programming environment often uses multiple cooperating processes to perform related operations. These processes must communicate with each other and share resources and information.
- The Kernel must provide mechanisms that make this possible. These mechanisms are collectively referred to as interprocess communication.
- Distributed computing systems make use of these facilities/resources to provide an Application Programming Interface (API) that allows IPC to be programmed at a higher level of abstraction (e.g., send and receive).

Five Types of Interprocess Communication

1. **Shared Memory**:
    - Allows processes to communicate by reading and writing to a specified memory location.
2. **Mapped Memory**:
    - Similar to shared memory, but it is associated with a file in the file system.
3. **Pipes**:
    - Enable sequential communication from one process to a related process.
4. **FIFOs**:
    - Similar to pipes, but allow unrelated processes to communicate by giving the pipe a name in the file system.
5. **Sockets**:

- Support communication between unrelated processes, even on different computers

Here is the information in a table format:

Name	Description	Scope	Use
File	Data is written to and read from a typical UNIX file.	Local	Sharing large data sets. Any number of processes can interoperate.
Pipe	Data is transferred between two processes using dedicated file descriptors.	Local	Simple data sharing, such as producer and consumer. Communication occurs only between a parent and child process.
Named Pipe	Data is exchanged between processes via dedicated file descriptors.	Local	Communication between any two peer processes on the same host. Used for producer-consumer or command-and-control, as in MySQL server and its CLI utility.
Signal	An interrupt alerts the application to a specific condition.	Local	Cannot transfer data in a signal; mostly used for process management.
Shared Memory	Information is shared by reading and writing from a common segment of memory.	Local	Cooperative work of any kind, especially if security is required.
Socket	After special setup, data is transferred using common input/output operations.	Local or Remote	Used in network services such as FTP, SSH, and the Apache web server.

Purposes of IPC

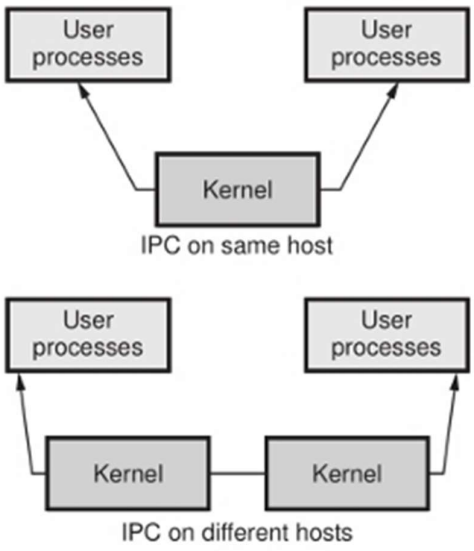
1. **Data transfer**
    - One process may wish to send data to another process.
2. **Sharing data**

- Multiple processes may wish to operate on shared data, such that if a process modifies the data, that change will be immediately visible to other processes sharing it.

3. **Event notification**
    - A process may wish to notify another process or set of processes that some event has occurred.
4. **Resource sharing**
    - The Kernel provides default semantics for resource allocation; they are not suitable for all applications.
5. **Process control**
    - A process such as a debugger may wish to assume complete control over the execution of another process.

Forms of IPC

- **IPC on the same host**
    - Involves communication between user processes on the same system.
    - The Kernel facilitates the interaction between processes.
- **IPC on different hosts**
    - Involves communication between processes on different systems.
    - The Kernel on each system manages the interaction between user processes.



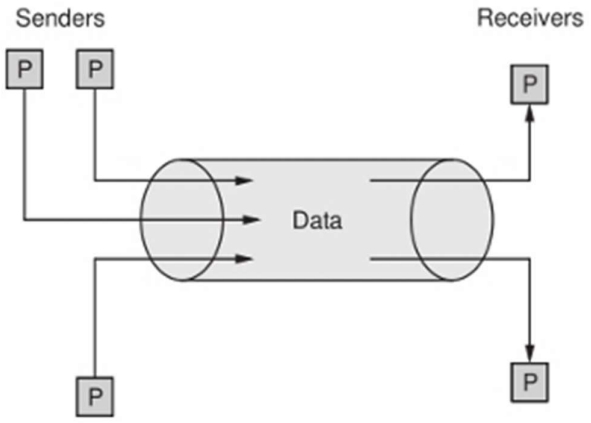
Functions of IPC

1. **Synchronization**
    - Used to coordinate access to resources among processes and also to coordinate the execution of these processes.
    - Methods include:
        - Record locking
        - Semaphores
        - Mutexes
        - Condition variables
2. **Message passing**
    - Used when processes wish to exchange information.
    - Message passing takes several forms, such as:
        - Pipes
        - FIFOs
        - Message queues
        - Shared memory

Pipes

- A pipe is a unidirectional, first-in first-out (FIFO) data stream of fixed maximum size that allows communication between processes. Writers add data to the end of the pipe, while readers retrieve data from the front. Once the data is read, it is removed and cannot be accessed again, ensuring that information flows in a structured manner.
- Pipes provide a flow control mechanism that synchronizes communication between processes. If a process attempts to read from an empty pipe, it is blocked until data is available. Similarly, if a process tries to write to a full pipe, it is blocked until another process reads the data, allowing for proper coordination between processes.
- A pipe is created using the pipe() system call, which returns two file descriptors—one for reading and the other for writing. These file descriptors are inherited by child processes, allowing them to share access to the same pipe. This mechanism enables efficient interprocess communication by establishing a direct data exchange between related processes.

- In terms of process communication, a pipe can have multiple readers and writers, meaning that different processes can read and write from the same pipe. A process may function as either a reader, a writer, or both, depending on the configuration of the file descriptors. However, one limitation of pipes is that they can only be used between processes with a common ancestor, such as a parent and child process.
- Typically, a parent process creates a pipe and then calls fork() to generate a child process. The pipe is then utilized for communication between the parent and child, allowing data to be passed efficiently. Pipes are widely used in interprocess communication (IPC) and are commonly implemented in command-line operations, such as using the pipe (|) operator in Unix-based systems to pass the output of one command as the input to another.



Properties of Pipes

1. Pipes do not have a name.
    - For this reason, the processes must share a parent process.
    - This is the main drawback of pipes.
    - However, pipes are treated as file descriptors, so they remain open even after fork and exec.
2. Pipes do not distinguish between messages; they just read a fixed number of bytes.
3. Pipes can also be used to get the output of a command or to provide input to a command.
4. The most common use of pipes is to let the output of one program become the input for another.
    - Users typically join two programs by a pipe using the shell's pipe operator (|).

Limitations of Pipes

1. Reading data removes it from the pipe, so a pipe cannot be used to broadcast data to multiple receivers.
2. Data in a pipe is treated as a byte stream and has no knowledge of message boundaries.
3. If there are multiple readers on a pipe, a writer cannot direct data to a specific reader.

Pipes Program:

The provided program demonstrates Interprocess Communication (IPC) using pipes where the parent process writes data, and the child process reads and prints it. However, there are a few issues with the code:

Code Explanation:

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#define MAXLINE 100  // Define max buffer size


int main(void) {

   int n;

   int fd[2];  // Pipe file descriptors

   pid_t pid;

   char line[MAXLINE];


   // Create a pipe

   if (pipe(fd) < 0) {

      perror("Pipe error");

      exit(1);

   }


   // Fork a new process
```

```
    if ((pid = fork()) < 0) {
        perror("Fork error");
        exit(1);
    }
    else if (pid > 0) {
        // Parent process
        close(fd[0]);  // Close unused read end
        write(fd[1], "hello world\n", 12);
        close(fd[1]);  // Close write end after writing
    }
    else {
        // Child process
        close(fd[1]);  // Close unused write end
        n = read(fd[0], line, MAXLINE);
        write(STDOUT_FILENO, line, n); // Print to stdout
        close(fd[0]);  // Close read end
    }

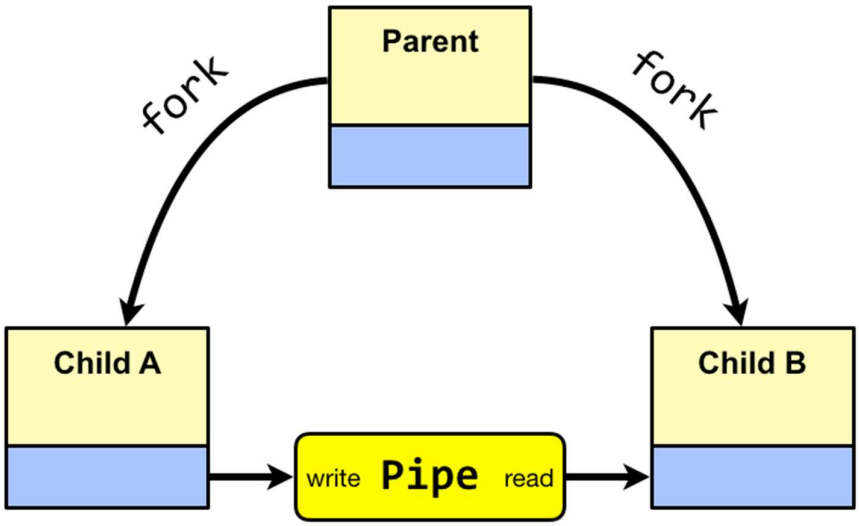
    return 0;
}
```

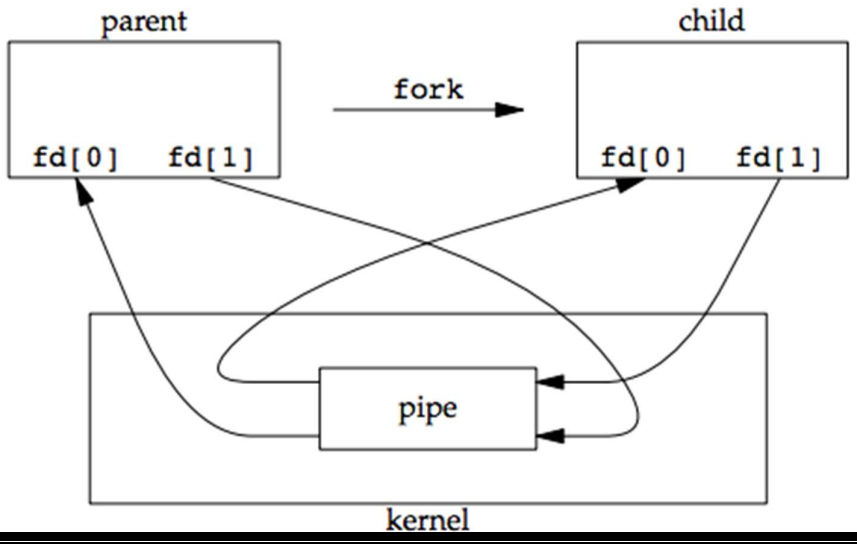
Explanation

1. **Creating a Pipe**:
    - pipe(fd) creates a pipe with:
        - fd[0] → Read end
        - fd[1] → Write end
2. **Forking a Child Process**:
    - fork() creates a new process.

- The pid value determines if it's the **parent** or **child**.

3. **Parent Process (pid > 0)**:
   - Closes **read end (fd[0])**.
   - Writes "hello world\n" to the pipe.
   - Closes **write end (fd[1])** after writing.
4. **Child Process (pid == 0)**:
   - Closes **write end (fd[1])**.
   - Reads data from fd[0] into line buffer.
   - Writes the received data to **standard output** (STDOUT_FILENO).
   - Closes **read end (fd[0])**.





Shared Memory

Shared Memory

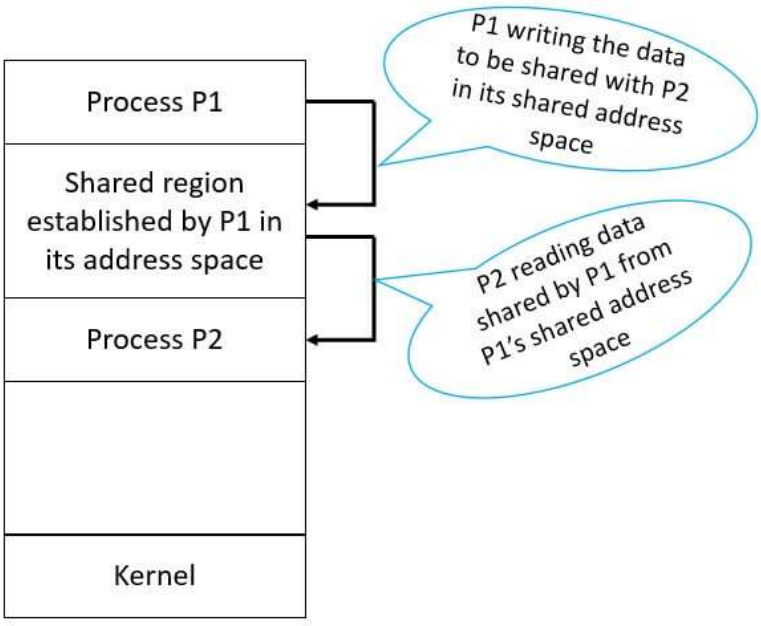
- A region of memory that is shared by cooperating processes is established.
- Processes exchange information by reading and writing data to the shared region.
- Shared memory allows maximum speed and convenience of communication, as it operates at memory speeds within a computer.
- Shared memory is faster than message passing because message-passing systems require system calls, which involve Kernel intervention.
- In contrast, shared-memory systems require system calls only to establish shared-memory regions.
- Once shared memory is established, all accesses are treated as routine memory accesses, and no Kernel assistance is required.

Advantages

1. Good for sharing large amounts of data.
2. Very fast.

Limitations

1. No synchronization is provided; applications must create their own synchronization mechanisms.
2. An alternative to shared memory is the mmap system call, which maps a file into the address space of the caller.



Shared Memory System

Deadlock

Deadlock in Semaphores

- The implementation of a semaphore with a waiting queue may result in a situation where two or more processes wait indefinitely for an event that can be caused only by one of the waiting processes.
- The event in question is the execution of a signal() operation.
- When such a state is reached, these processes are said to be **deadlocked**.

Illustration of Deadlock

- Consider a system with two processes, **P0** and **P1**, each accessing two semaphores, **S** and **Q**, both initialized to 1:

Process P0

wait(S);

wait(Q);

...

signal(S);

signal(Q);

Process P1

wait(Q);

wait(S);

...

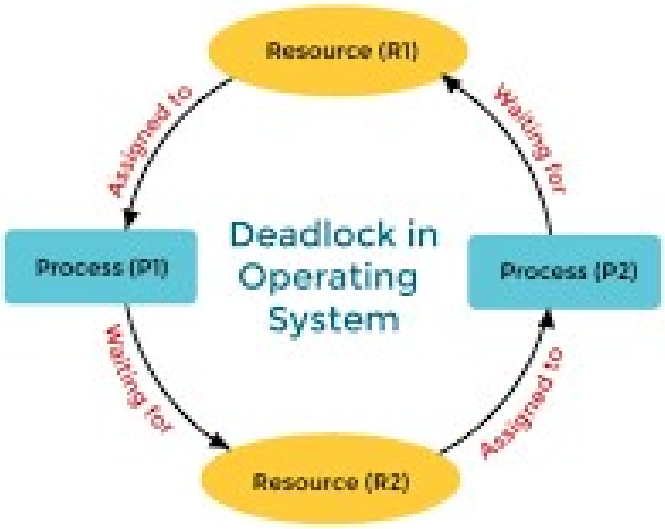
signal(Q);

signal(S);

- Suppose **P0** executes wait(S), then **P1** executes wait(Q).
- When **P0** executes wait(Q), it must wait until **P1** executes signal(Q).
- Similarly, when **P1** executes wait(S), it must wait until **P0** executes signal(S).
- Since these signal() operations cannot be executed, **P0** and **P1** are **deadlocked**.

Definition of Deadlock

- A set of processes is in a **deadlocked state** when every process in the set is waiting for an event that can be caused only by another process in the set.
- The **events** mainly concerned here are the acquisition and release of resources such as **mutex locks** and **semaphores**.
- Various mechanisms exist for dealing with the **deadlock problem** and other forms of **liveness failures**.
- A lack of process synchronization can result in two extreme conditions are deadlock or starvation. Deadlock is the problem of multiprogrammed system.
- Deadlock can be defined as the permanent blocking of a set of processes that either complete for system resources. Deadlock is more serious than starvation.
- Deadlock can occur on sharable resources such as files, printers, database, disks, tape drives, memory, CPU cycles etc.
- A process is in deadlock state if it was waiting for a particular event that will not occur. In a system deadlock, one or more processes are deadlocked.



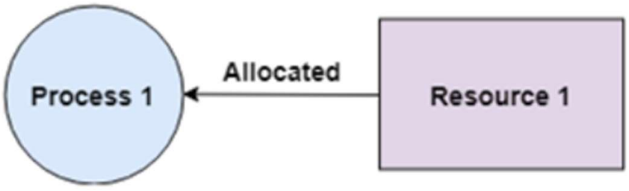
Deadlock Characterization

Necessary Conditions for Deadlock:

For a **deadlock** to occur, the following **four conditions** must hold simultaneously:

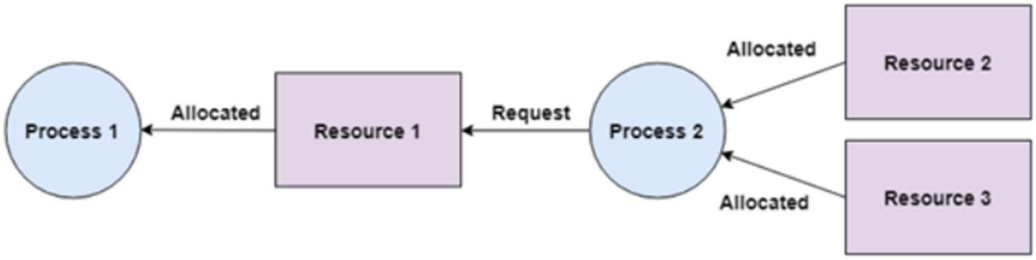
1. Mutual Exclusion

- **Definition:** A resource can only be used by **one process at a time**, meaning no two processes can access the resource simultaneously.
- **Explanation:**
    - If multiple processes attempt to use the same resource at the same time, **data inconsistency** or system crashes may occur.
    - To prevent this, the system enforces mutual exclusion, allowing **only one process** to access the resource at a given time.
- **Example:**
    - A printer in a system cannot be shared between two processes at the same time. If Process P1 is printing a document, Process P2 must wait until P1 releases the printer.



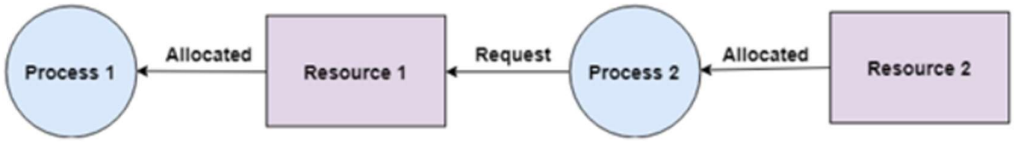
2. Hold and Wait

- **Definition:** A process holding at least **one resource** can **request additional resources** while still holding the currently allocated resources.
- **Explanation:**
    - This condition allows processes to **wait indefinitely** for more resources, which can lead to deadlock if the requested resources are held by other waiting processes.
    - It **creates dependency** between processes, leading to a **chain of waiting** if multiple processes request the same resources.
- **Example:**
    - Process P1 holds Resource R1 and requests R2. At the same time, Process P2 holds R2 and requests R1. Both processes are now **waiting for each other**, leading to deadlock.



3. No Preemption

- **Definition:** A resource **cannot be forcibly taken** from a process; it must be **released voluntarily** by the process holding it.
- **Explanation:**
  - If a process is holding a resource and does not voluntarily release it, other processes waiting for that resource **remain blocked indefinitely**.
  - Some systems implement **preemptive resource allocation** to avoid deadlocks, but many systems do not allow forced removal due to data consistency concerns.
- **Example:**
  - A process writing data to a file cannot be interrupted in the middle of the operation. If another process requests access to the same file, it must wait until the first process finishes and releases the file.

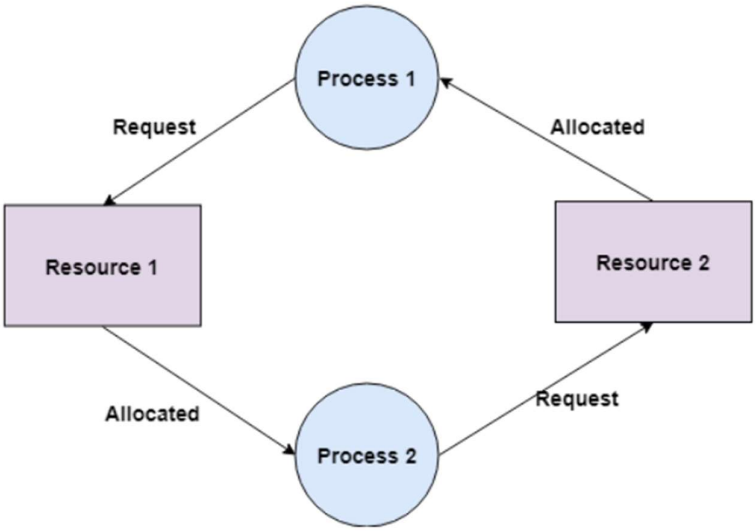


4. Circular Wait

- **Definition:** A set of processes exist in a **circular chain**, where **each process is waiting for a resource held by the next process in the cycle**.
- **Explanation:**

- The **circular dependency** creates a scenario where **no process can proceed** because each one is waiting for another process in the cycle to release a resource.
- This is the **final step** in forming a deadlock, as it **traps processes in an infinite wait**.

- **Example:**
  - Suppose there are **three processes (P1, P2, P3)** and **three resources (R1, R2, R3)**:
    - P1 holds R1 and waits for R2.
    - P2 holds R2 and waits for R3.
    - P3 holds R3 and waits for R1.
    - This forms a **circular wait condition**, leading to **deadlock**.



Overall:

- All four conditions must hold simultaneously for a deadlock to occur.
- If any one condition is removed, deadlock cannot happen.
- Deadlocks must be prevented or handled using techniques like resource allocation graphs, deadlock detection, and recovery strategies.

Methods for Handling Deadlocks

There are four primary approaches to handling deadlocks in an operating system:

1. Deadlock Prevention

- **Definition:** Deadlock prevention ensures that the system **never enters a deadlock state** by removing one or more of the necessary conditions for deadlock.
- **Explanation:**
    - Since deadlock occurs when all four necessary conditions (Mutual Exclusion, Hold and Wait, No Preemption, Circular Wait) are present, preventing at least one of them will ensure deadlocks do not occur.
    - However, this approach may lead to **poor resource utilization** and may not always be efficient.
- **Techniques for Prevention:**
    - **Eliminate Mutual Exclusion:** Some resources (e.g., read-only files) can be shared among multiple processes.
    - **Eliminate Hold and Wait:** Processes must request all resources at the beginning, reducing resource utilization.
    - **Eliminate No Preemption:** If a process holding resources is waiting for additional resources, preempt and allocate them to other processes.
    - **Eliminate Circular Wait:** Impose an ordering of resource allocation to prevent circular waiting.
- **Disadvantages:**
    - Leads to **inefficient resource utilization** since processes may be forced to request all resources in advance.
    - May **increase waiting time** for processes that need multiple resources.

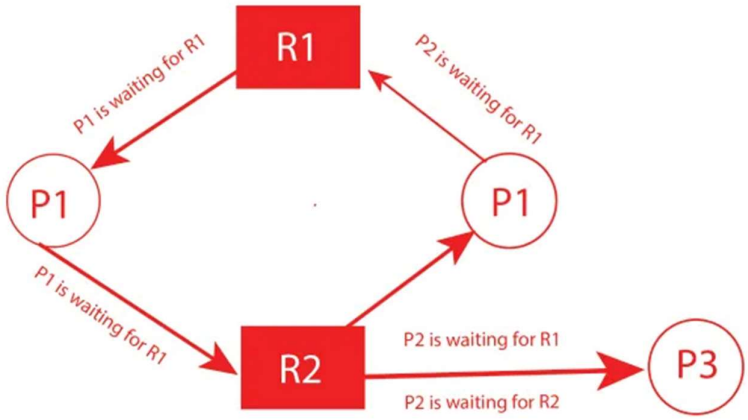
Explanation

- **Processes (P1, P2, P3)** represented by circles.
- **Resources (R1, R2)** represented by squares.
- **Edges (Arrows)** showing request and allocation of resources:
    - **Request Edge (→)**: A process is waiting for a resource.

- **Assignment Edge (←)**: A process is holding a resource.

From the diagram:

- **P1 is waiting for R1**, but R1 is held by another process.
- **P2 is waiting for both R1 and R2**, which are already held by other processes.
- **Circular Wait Condition Exists**: Since **P1 → R1 → P2 → R2 → P1**, a deadlock has occurred.



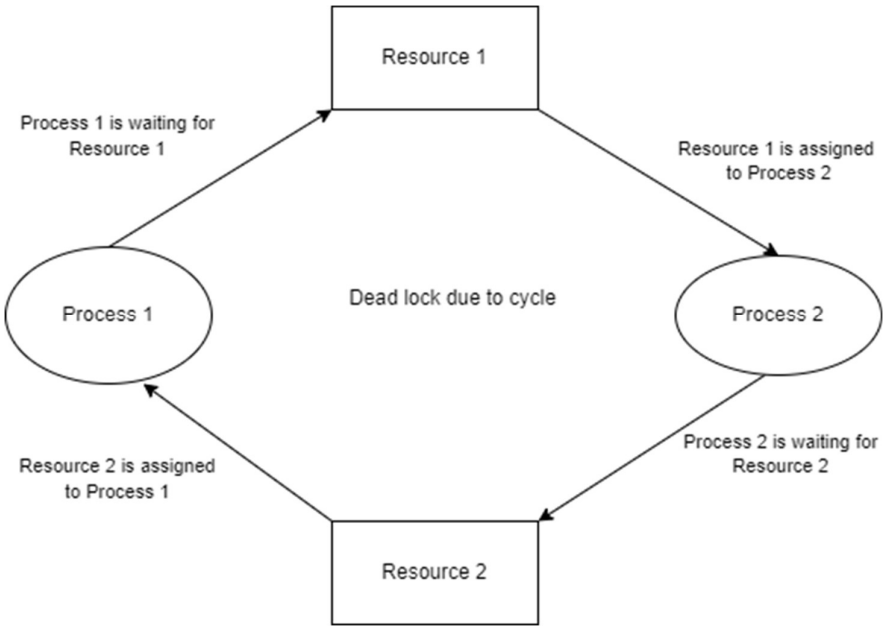
2. Deadlock Avoidance

- **Definition:** Deadlock avoidance ensures the system **only moves to safe states** and never enters an unsafe state where deadlock is possible.
- **Explanation:**
  - Unlike deadlock prevention, deadlock avoidance **does not eliminate** necessary conditions but **manages resource allocation** dynamically to avoid deadlock.
  - The system must have **prior knowledge** of how resources are requested and released.
- **Techniques for Avoidance:**
  - **Resource Allocation Graph (RAG):**
    - The system maintains a graph showing the relationship between processes and resources.

- If allocating a resource leads to a cycle (unsafe state), the request is denied.

- **Banker's Algorithm:**
  - Used for multiple instances of resources.
  - The system checks if granting a resource request keeps it in a safe state.
  - If the system remains in a safe state, the request is granted; otherwise, the process must wait.

- **Disadvantages:**
  - Requires prior knowledge of **maximum resource needs** of each process.
  - **Complex implementation** in large systems.



3. Deadlock Detection

- **Definition:** This method allows **deadlocks to occur** but periodically **checks** for them and takes corrective action.
- **Explanation:**

- Used in systems where **deadlocks cannot be completely prevented or avoided**.
- The system **monitors resource allocation** and periodically runs an algorithm to check if a deadlock exists.
- If a deadlock is found, the system takes appropriate actions to **resolve it**.

- **Techniques for Detection:**
  - **Wait-for Graph:**
    - A **simplified version of the Resource Allocation Graph (RAG)** used for detecting deadlocks.
    - If the graph has **a cycle**, a deadlock has occurred.
  - **Deadlock Detection Algorithm:**
    - Similar to Banker's Algorithm but used to **identify** deadlocked processes.
- **Disadvantages:**
  - **High overhead** due to continuous monitoring.
  - **Deadlock resolution** can lead to **process termination or resource preemption**.

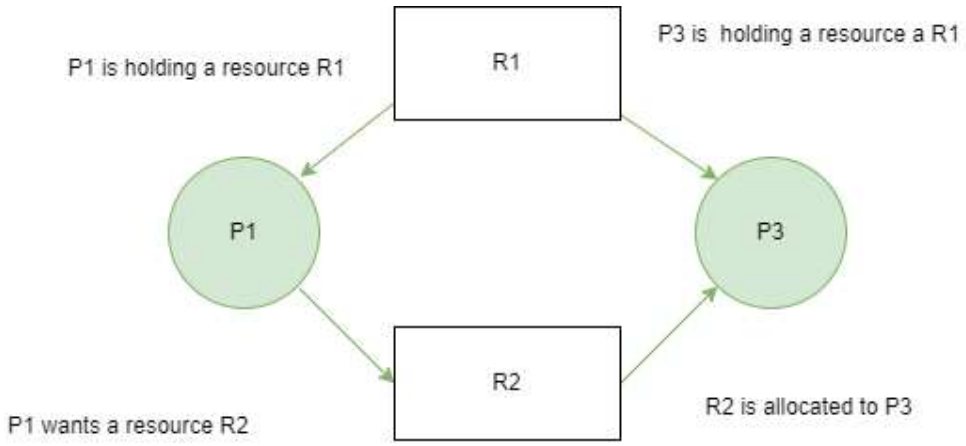
4. Deadlock Recovery

- **Definition:** If a deadlock is detected, **recovery techniques** are used to remove it and allow processes to continue execution.
- **Explanation:**
  - The system **identifies the deadlocked processes** and takes steps to **break the deadlock** by preempting resources or terminating processes.
- **Techniques for Recovery:**
  - **Process Termination:**
    - Terminate **one or more processes** involved in deadlock to release resources.
    - Can be done in two ways:

1. **Abort all deadlocked processes:** Ensures deadlock is resolved but results in **loss of work**.
2. **Abort one process at a time until deadlock is resolved:** Requires extra computations to determine which process to terminate.

- **Resource Preemption:**
  - The system forcibly **takes resources from processes** to break the deadlock.
  - Requires mechanisms to **rollback** affected processes.

- **Disadvantages:**
  - Terminating processes may **cause loss of progress**.
  - Resource preemption can lead to **inconsistent system states**.



Comparison of Deadlock Avoidance, Detection, and Prevention :

Parameters	Avoidance	Detection	Prevention
1. Resource Allocation Policy	Midway between detection and prevention.	Very liberal, allocates resources freely.	Conservative, commits resources beforehand.

2. Strategy Used	Ensures at least one safe path to avoid deadlock.	Periodically checks for deadlocks.	Prevents deadlock by eliminating one of the necessary conditions.
3. Implementation Complexity	Moderate complexity as safe states must be identified.	Simple to implement but requires periodic checking.	Complex due to strict resource allocation policies.
4. System Overhead	Moderate, as resource allocation decisions are made dynamically.	High, as it requires deadlock detection algorithms.	High, as it requires strict conditions to prevent deadlock.
5. Process Blocking	Processes may be blocked for a long time if no safe path exists.	Processes are not blocked but may need to be terminated upon detection.	Processes may be delayed at the start to prevent deadlocks.
6. Preemption Requirement	No preemption required.	Preemption may be necessary to resolve deadlocks.	May use preemption as a strategy to avoid deadlocks.
7. Suitable for	Systems where safe states can be determined dynamically.	Systems where deadlocks are rare but must be resolved when they occur.	Systems where strict policies can be enforced to prevent deadlocks.
8. Disadvantages	May lead to unnecessary blocking of processes.	Preemption losses and rollback overhead.	Delays process initiation and may lead to underutilization of resources.